ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA CÔNG NGHỆ THÔNG TIN

CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI

XÂY DỰNG HỆ THỐNG MICROSERVICES

PRODUCT – INVENTORY – ORDER SERVICES

Giảng viên hướng dẫn:	TS Đỗ Như Tài
Sinh viên :	**Mssv:**
Nguyễn Việt Đức	3122411044
Nguyễn Huỳnh Phương Lộc	3122411117
Nguyễn Phạm Hoàng Quân	3122411169
Nguyễn Trần Quốc Tiến	3122411208

Thành phố Hồ Chí Minh, tháng 3 năm 2026

LỜI CẢM ƠN

Trong suốt quá trình học tập, nghiên cứu và thực hiện đề tài **"Xây dựng hệ thống Microservices với Product Service, Inventory Service và Order Service"**, nhóm chúng em đã nhận được rất nhiều sự quan tâm, hướng dẫn và hỗ trợ quý báu từ giảng viên, gia đình và bạn bè. Đây là nguồn động viên rất lớn để nhóm chúng em có thể hoàn thành bài báo cáo này.

Trước hết, nhóm chúng em xin bày tỏ lòng biết ơn chân thành và sâu sắc đến quý thầy cô của khoa đã tận tình giảng dạy, truyền đạt cho nhóm chúng em những kiến thức nền tảng và chuyên môn quan trọng trong suốt quá trình học tập. Những kiến thức đó chính là cơ sở để nhóm chúng em có thể tiếp cận, phân tích và triển khai đề tài về kiến trúc microservices một cách hệ thống và thực tế hơn.

Đặc biệt, nhóm chúng em xin gửi lời cảm ơn chân thành đến giảng viên hướng dẫn, người đã trực tiếp định hướng, góp ý và hỗ trợ nhóm chúng em trong quá trình thực hiện đề tài. Những nhận xét, chỉ dẫn và góp ý của thầy, cô không chỉ giúp nhóm chúng em hoàn thiện bài báo cáo mà còn giúp nhóm chúng em hiểu rõ hơn về cách tư duy, tổ chức và triển khai một hệ thống phần mềm hiện đại.

Nhóm chúng em cũng xin cảm ơn gia đình đã luôn là chỗ dựa tinh thần vững chắc, luôn động viên và tạo điều kiện thuận lợi để các thành viên trong nhóm có thể yên tâm học tập và hoàn thành nhiệm vụ của mình. Bên cạnh đó, nhóm chúng em xin cảm ơn bạn bè, anh chị và những người đã hỗ trợ nhóm trong việc trao đổi kiến thức, chia sẻ tài liệu và góp ý trong quá trình thực hiện đề tài.

Mặc dù nhóm chúng em đã cố gắng hoàn thiện bài báo cáo với tinh thần nghiêm túc và cầu thị, song do kiến thức và kinh nghiệm thực tế còn hạn chế nên bài làm khó tránh khỏi những thiếu sót. Nhóm chúng em rất mong nhận được sự đóng góp ý kiến từ quý thầy cô để bài báo cáo được hoàn thiện hơn, đồng thời giúp các thành viên tích lũy thêm kinh nghiệm cho quá trình học tập và làm việc sau này.

Một lần nữa, nhóm chúng em xin chân thành cảm ơn.

PHÂN CÔNG CÔNG VIỆC

STT	Họ và tên	MSSV	Phân công công việc
1	Nguyễn Phạm Hoàng Quân	3122411169	Chương 1
2	Nguyễn Trần Quốc Tiến	3122411208	Chương 2
3	Nguyễn Việt Đức	3122411044	Chương 3 và Chương 4
4	Nguyễn Huỳnh Phương Lộc	3122411117	Chương 5 và Chương 6

MỤC LỤC

LỜI CẢM ƠN **1**
PHÂN CÔNG CÔNG VIỆC **2**
LỜI MỞ ĐẦU **4**
CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI **6**
1.1. Lý do chọn đề tài 6
1.2. Mục tiêu nghiên cứu 6
1.3. Đối tượng và phạm vi nghiên cứu 7
1.4. Phương pháp thực hiện 8
1.5. Cấu trúc báo cáo 8
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT **9**
2.1. Tổng quan về kiến trúc Microservices 9
2.1.1. Ưu điểm của Microservices 9
2.1.2. Hạn chế của Microservices 10
2.2. Giới thiệu về Spring Boot 10
2.3. RESTful API 10
2.4. Cơ sở dữ liệu trong Microservices 11
2.5. Eureka Discovery Server 11
2.6. Resilience4j 12
2.7. Kafka 12
2.8. Actuator, Prometheus và Zipkin 13
2.9. Docker và Docker Compose 13
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG **14**
3.1. Phân tích yêu cầu bài toán 14
3.1.1. Yêu cầu đối với Product Service 14
3.1.2. Yêu cầu đối với Inventory Service 14
3.1.3. Yêu cầu đối với Order Service 14
3.2. Phân tích mối quan hệ giữa các service 15
3.3. Thiết kế kiến trúc tổng thể 15
3.4. Phân tích luồng nghiệp vụ 16
3.4.1. Luồng thêm sản phẩm 16
3.4.2. Luồng lấy danh sách sản phẩm 16
3.4.3. Luồng kiểm tra tồn kho 16
3.4.4. Luồng đặt hàng 16
3.5. Thiết kế dữ liệu 17
3.5.1. Dữ liệu của Product Service 17
3.5.2. Dữ liệu của Inventory Service 18
3.5.3. Dữ liệu của Order Service 19
3.6. Thiết kế lớp và cấu trúc dự án 20
CHƯƠNG 4. XÂY DỰNG HỆ THỐNG **21**
4.1. Môi trường và công nghệ sử dụng 21
4.2. Xây dựng Product Service 21
4.2.1. Vai trò của Product Service 21

4.2.2. Thiết kế API của Product Service....21
4.2.3. Mô hình dữ liệu của Product Service....21
4.2.4. Tổ chức mã nguồn Product Service....22
4.3. Xây dựng Inventory Service....22
4.3.1. Vai trò của Inventory Service....22
4.3.2. Thiết kế API kiểm tra tồn kho....22
4.3.3. Mô hình dữ liệu Inventory....22
4.3.4. Sử dụng Spring Data JPA....23
4.3.5. Tích hợp Eureka trong Inventory Service....23
4.3.6. Tích hợp Actuator, Prometheus và Zipkin....23
4.3.7. Kiểm thử Inventory Service....23
4.4. Xây dựng Order Service....23
4.4.1. Vai trò của Order Service....23
4.4.2. Thiết kế API đặt hàng....23
4.4.3. Mô hình dữ liệu Order....24
4.4.4. Luồng xử lý đặt hàng chi tiết....24
4.4.5. Tích hợp Eureka trong Order Service....24
4.4.6. Tích hợp Kafka trong Order Service....24
4.4.7. Tích hợp Resilience4j trong Order Service....25
4.4.8. Tích hợp Actuator, Prometheus, Zipkin....25
4.4.9. Kiểm thử Order Service....26
4.5. Tích hợp Eureka Discovery Server....26
4.6. Docker hóa hệ thống....26
CHƯƠNG 5. KIỂM THỬ VÀ ĐÁNH GIÁ....28
5.1. Mục tiêu kiểm thử....28
5.2. Kiểm thử Product Service....28
5.3. Kiểm thử Inventory Service....28
5.4. Kiểm thử Order Service....28
5.5. Unit test và Integration test....29
5.6. Đánh giá kết quả đạt được....29
5.7. Ưu điểm và hạn chế của hệ thống....29
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN....31
6.1. Kết luận....31
6.2. Hướng phát triển....31

LỜI MỞ ĐẦU

Trong bối cảnh các hệ thống phần mềm ngày càng phát triển về quy mô và độ phức tạp, việc lựa chọn kiến trúc phù hợp đóng vai trò rất quan trọng. Kiến trúc microservices đã trở thành một xu hướng phổ biến nhờ khả năng phân tách hệ thống thành các dịch vụ nhỏ, độc lập, giúp dễ dàng mở rộng, bảo trì và triển khai.

Xuất phát từ thực tế đó, nhóm chúng em thực hiện đề tài **"Xây dựng hệ thống Microservices với Product Service, Inventory Service và Order Service"** nhằm nghiên cứu và áp dụng kiến trúc microservices vào bài toán quản lý bán hàng.

Trong đề tài này, hệ thống được chia thành ba dịch vụ chính:

- **Product Service**: quản lý thông tin sản phẩm
- **Inventory Service**: kiểm tra và quản lý tồn kho
- **Order Service**: xử lý nghiệp vụ đặt hàng

Thông qua việc xây dựng các service trên, nhóm chúng em tiếp cận và thực hành các công nghệ như Spring Boot, Eureka, Kafka, Resilience4j và Docker, đồng thời hiểu rõ hơn về cách thiết kế và triển khai một hệ thống microservices.

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1. Lý do chọn đề tài

Trong những năm gần đây, cùng với sự phát triển mạnh của các hệ thống phần mềm phân tán, yêu cầu đặt ra đối với một ứng dụng hiện đại không chỉ dừng lại ở việc đáp ứng đúng chức năng, mà còn phải bảo đảm khả năng mở rộng, bảo trì, triển khai nhanh và chịu lỗi tốt. Với các hệ thống nhỏ, kiến trúc nguyên khối (monolithic) có thể đáp ứng khá tốt do toàn bộ chức năng được xây dựng tập trung trong một ứng dụng duy nhất. Tuy nhiên, khi quy mô hệ thống tăng lên, số lượng chức năng nhiều hơn, số người dùng lớn hơn và nhu cầu thay đổi liên tục, mô hình monolithic bắt đầu bộc lộ những hạn chế rõ rệt.

- **Hạn chế đầu tiên của mô hình nguyên khối là tính phụ thuộc cao giữa các thành phần.** Khi toàn bộ chức năng nằm trong cùng một khối ứng dụng, việc sửa đổi một chức năng nhỏ cũng có thể ảnh hưởng đến toàn bộ hệ thống.
- **Hạn chế thứ hai là khó mở rộng linh hoạt.** Trong thực tế, không phải chức năng nào cũng có mức tải như nhau. Ví dụ, trong một hệ thống bán hàng, chức năng đặt hàng có thể chịu tải lớn hơn so với quản trị danh mục sản phẩm. Nếu sử dụng monolithic, nhà phát triển thường phải mở rộng toàn bộ hệ thống, gây lãng phí tài nguyên.
- Ngoài ra, việc triển khai, kiểm thử và gỡ lỗi trong hệ thống nguyên khối thường mất nhiều thời gian, nhất là khi số lượng module lớn.

Để khắc phục các nhược điểm trên, kiến trúc microservices được xem là một hướng tiếp cận phù hợp. Microservices cho phép chia hệ thống thành các dịch vụ nhỏ, mỗi dịch vụ đảm nhận một chức năng nghiệp vụ riêng, có thể triển khai độc lập, bảo trì độc lập và mở rộng độc lập. Cách tiếp cận này đặc biệt phù hợp với các hệ thống thương mại điện tử, bán hàng trực tuyến và quản lý kho, nơi các nghiệp vụ như quản lý sản phẩm, kiểm tra tồn kho, xử lý đơn hàng hay gửi thông báo có tính độc lập tương đối.

Xuất phát từ nhu cầu đó, đề tài "Xây dựng hệ thống Microservices gồm Product Service, Inventory Service và Order Service" được lựa chọn nhằm nghiên cứu cách tổ chức một hệ thống backend hiện đại dựa trên Spring Boot và Spring Cloud. Đây là một bài toán có tính thực tiễn cao, vì ba dịch vụ được lựa chọn đại diện cho ba nghiệp vụ cốt lõi trong hầu hết các hệ thống bán hàng hiện nay. Thông qua đề tài này, người thực hiện có cơ hội tiếp cận từ lý thuyết đến thực hành về kiến trúc microservices, cơ chế giao tiếp giữa các dịch vụ, tổ chức dữ liệu, theo dõi hệ thống, xử lý lỗi và triển khai ứng dụng.

1.2. Mục tiêu nghiên cứu

Mục tiêu tổng quát của đề tài là xây dựng thành công một hệ thống microservices cơ bản nhưng đầy đủ các thành phần thiết yếu của một hệ thống phân tán hiện đại. Hệ thống này bao gồm ba dịch vụ: Product Service, Inventory Service và Order Service, trong đó mỗi service đảm nhận một vai trò riêng biệt và phối hợp với nhau để hoàn thành quy trình xử lý đặt hàng.

Cụ thể, đề tài hướng đến các mục tiêu sau:

- **Thứ nhất,** xây dựng Product Service với nhiệm vụ lưu trữ thông tin sản phẩm và cung cấp API để tạo mới cũng như truy vấn danh sách sản phẩm. Theo yêu cầu bài toán, Product Service sử dụng mô hình dữ liệu gồm các trường id, name, description, price.
- **Thứ hai,** xây dựng Inventory Service với nhiệm vụ quản lý thông tin kho hàng, hỗ trợ API kiểm tra trạng thái tồn kho dựa trên danh sách mã sản phẩm skuCode. Dịch vụ này sử dụng mô hình dữ liệu gồm id, skuCode, quantity, đồng thời phải tích hợp Eureka, Actuator, Prometheus và Zipkin.
- **Thứ ba,** xây dựng Order Service với chức năng tiếp nhận yêu cầu đặt hàng, kiểm tra tồn kho thông qua Inventory Service, lưu đơn hàng khi điều kiện hợp lệ và phát sinh sự kiện thông báo thông qua Kafka. Ngoài ra, Order Service còn phải áp dụng Resilience4j để tăng tính ổn định khi gọi sang Inventory Service. Mô hình dữ liệu gồm Order và OrderLineItems, trong đó Order có id, orderNumber, còn OrderLineItems có id, skuCode, price, quantity.
- **Bên cạnh đó,** đề tài còn hướng tới mục tiêu giúp người thực hiện rèn luyện kỹ năng thiết kế kiến trúc hệ thống, xây dựng REST API, tích hợp nhiều công nghệ phổ biến trong hệ sinh thái Spring, viết kiểm thử, đóng gói ứng dụng bằng Docker và đánh giá toàn bộ hệ thống dưới góc nhìn kỹ thuật.

1.3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu của đề tài là kiến trúc microservices và các công nghệ phục vụ việc xây dựng một hệ thống backend phân tán. Trong phạm vi thực hiện, đề tài tập trung chủ yếu vào tầng backend, cụ thể là việc xây dựng các dịch vụ độc lập, tổ chức dữ liệu, giao tiếp giữa các service, xử lý lỗi và theo dõi hệ thống.

Phạm vi chức năng của hệ thống được giới hạn ở ba nghiệp vụ chính:

- **Nghiệp vụ thứ nhất** là quản lý sản phẩm, do Product Service đảm nhiệm.
- **Nghiệp vụ thứ hai** là kiểm tra tồn kho, do Inventory Service đảm nhiệm.
- **Nghiệp vụ thứ ba** là xử lý đơn hàng, do Order Service đảm nhiệm.

Các thành phần khác như giao diện người dùng, quản lý người dùng, thanh toán điện tử hoặc vận chuyển không nằm trong phạm vi chính của đề tài.

Về phạm vi công nghệ, đề tài sử dụng Spring Boot để xây dựng dịch vụ, Spring Data JPA hoặc Spring Data tương ứng với từng service để thao tác dữ liệu, Eureka để discovery service, Kafka để truyền thông điệp, Resilience4j để xử lý lỗi, và Docker để đóng gói triển khai. Ngoài ra, đề tài còn có xét đến giám sát hệ thống thông qua Actuator, Prometheus và Zipkin ở các service được yêu cầu.

1.4. Phương pháp thực hiện

Đề tài được thực hiện theo quy trình gồm nhiều bước:

1. **Giai đoạn nghiên cứu yêu cầu:** Nội dung từng đề bài riêng cho Product Service, Inventory Service và Order Service được phân tích để xác định chính xác chức năng, mô hình dữ liệu, API cần xây dựng và các công nghệ tích hợp.
2. **Giai đoạn nghiên cứu lý thuyết:** Tập trung vào microservices, RESTful API, service discovery, message broker, fault tolerance và monitoring.
3. **Giai đoạn phân tích và thiết kế hệ thống:** Ở giai đoạn này, người thực hiện xác định kiến trúc tổng thể, luồng dữ liệu giữa các service, cấu trúc thư mục của từng dự án, sơ đồ lớp, sơ đồ tuần tự và mối quan hệ giữa các thành phần.
4. **Giai đoạn hiện thực:** Sau khi có thiết kế, hệ thống được hiện thực bằng Spring Boot. Mỗi service được xây dựng riêng, cấu hình database riêng, API riêng và tích hợp các thành phần mở rộng theo đúng yêu cầu.
5. **Giai đoạn kiểm thử và triển khai:** Sau khi hoàn thiện phần lập trình, hệ thống được kiểm thử bằng các công cụ như Postman để xác minh API hoạt động đúng, đồng thời kiểm thử kịch bản liên service, đặc biệt là quy trình đặt hàng từ Order Service sang Inventory Service. Cuối cùng, các dịch vụ được đóng gói bằng Docker, từ đó tạo tiền đề cho việc triển khai đồng thời nhiều thành phần trong một môi trường nhất quán.

1.5. Cấu trúc báo cáo

Nội dung báo cáo được tổ chức thành sáu chương nhằm bảo đảm tính logic và dễ theo dõi:

- **Chương 1** trình bày lý do chọn đề tài, mục tiêu, phạm vi và phương pháp thực hiện.
- **Chương 2** trình bày cơ sở lý thuyết liên quan đến microservices và các công nghệ sử dụng.
- **Chương 3** phân tích yêu cầu và thiết kế hệ thống.
- **Chương 4** trình bày quá trình xây dựng từng service và tích hợp các thành phần kỹ thuật.
- **Chương 5** tập trung vào kiểm thử và đánh giá kết quả đạt được.
- **Chương 6** tổng kết kết quả thực hiện và đề xuất hướng phát triển trong tương lai.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về kiến trúc Microservices

Kiến trúc microservices là một phong cách thiết kế phần mềm trong đó ứng dụng được chia thành nhiều dịch vụ nhỏ, độc lập và tập trung quanh một chức năng nghiệp vụ cụ thể. Mỗi service có thể được phát triển, triển khai, cập nhật và mở rộng mà không cần phụ thuộc hoàn toàn vào các service khác. Điều này khác biệt đáng kể so với kiến trúc nguyên khối, nơi mọi chức năng của hệ thống đều nằm trong một khối ứng dụng duy nhất.

Bản chất của microservices là sự phân tách theo miền nghiệp vụ. Trong hệ thống của đề tài này, việc phân chia thành Product Service, Inventory Service và Order Service là minh họa điển hình cho nguyên tắc đó. Product Service chỉ quan tâm đến thông tin sản phẩm; Inventory Service chỉ tập trung vào số lượng tồn kho; Order Service chỉ tập trung vào quy trình đặt hàng. Chính sự tách biệt này giúp hệ thống trở nên rõ ràng hơn về trách nhiệm của từng thành phần.

Một đặc trưng quan trọng của microservices là mỗi service có thể có cơ sở dữ liệu riêng. Điều này giúp tránh tình trạng nhiều module cùng truy cập và phụ thuộc vào một cơ sở dữ liệu trung tâm, từ đó giảm ràng buộc giữa các thành phần. Đồng thời, mỗi service có thể lựa chọn công nghệ lưu trữ phù hợp nhất với nhu cầu. Trong thực tế, Product Service có thể dùng MongoDB hoặc một công nghệ lưu trữ tài liệu, trong khi Inventory Service và Order Service phù hợp với cơ sở dữ liệu quan hệ để xử lý giao dịch và tính toàn vẹn dữ liệu. Yêu cầu bài của Product Service nêu mô hình sản phẩm gồm id, name, description, price, còn Inventory Service và Order Service đều yêu cầu mô hình dữ liệu theo kiểu quan hệ, phù hợp với việc quản lý kho và đơn hàng.

2.1.1. Ưu điểm của Microservices

- **Ưu điểm nổi bật nhất của microservices là khả năng mở rộng độc lập.** Khi một service chịu tải lớn, chỉ cần mở rộng riêng service đó thay vì mở rộng toàn bộ hệ thống. Ví dụ, nếu hệ thống có nhiều yêu cầu đặt hàng, Order Service có thể được scale thêm nhiều instance mà không cần scale Product Service.
- **Ưu điểm thứ hai là dễ bảo trì.** Mỗi service có phạm vi chức năng rõ ràng, mã nguồn ngắn gọn hơn, dễ hiểu hơn. Đội ngũ phát triển cũng có thể chia việc theo service, từ đó tăng hiệu quả cộng tác.
- **Ưu điểm thứ ba là triển khai độc lập.** Một service có thể được cập nhật hoặc sửa lỗi mà không cần dừng toàn bộ hệ thống, miễn là hợp đồng giao tiếp giữa các service được giữ nguyên.
- **Ngoài ra, microservices còn giúp tăng tính chịu lỗi.** Nếu được thiết kế tốt, lỗi ở một service không nhất thiết làm sập toàn bộ hệ thống. Điều này đặc biệt

quan trọng trong đề tài, nơi Order Service gọi Inventory Service qua mạng và cần cơ chế như Resilience4j để bảo vệ luồng xử lý đặt hàng.

2.1.2. Hạn chế của Microservices

Bên cạnh ưu điểm, microservices cũng mang lại nhiều thách thức:

- **Trước hết là độ phức tạp trong quản lý.** Thay vì một ứng dụng duy nhất, hệ thống sẽ có nhiều service, nhiều cấu hình, nhiều database, nhiều cổng và nhiều cơ chế giao tiếp. Việc thiết lập môi trường phát triển và triển khai vì thế cũng khó hơn.
- **Hạn chế tiếp theo là khó gỡ lỗi.** Trong hệ thống phân tán, một request có thể đi qua nhiều service khác nhau. Khi lỗi xảy ra, việc xác định lỗi phát sinh ở đâu cần có công cụ hỗ trợ tracing như Zipkin. Đây cũng là lý do đề bài yêu cầu tích hợp Zipkin cho Inventory Service và Order Service.
- **Ngoài ra,** microservices đòi hỏi sự cẩn trọng trong thiết kế giao tiếp giữa các service, quản lý phiên bản API, đồng bộ dữ liệu và bảo đảm độ tin cậy của hệ thống. Nếu không có công cụ phù hợp như service discovery, message broker hoặc circuit breaker, hệ thống có thể trở nên mong manh và khó vận hành.

2.2. Giới thiệu về Spring Boot

Spring Boot là framework thuộc hệ sinh thái Spring, được thiết kế nhằm đơn giản hóa quá trình xây dựng các ứng dụng Java, đặc biệt là ứng dụng web và microservices. Trước đây, việc cấu hình một dự án Spring truyền thống đòi hỏi rất nhiều thao tác thủ công, từ khai báo bean, cấu hình server đến tổ chức các file XML phức tạp. Spring Boot ra đời nhằm giảm bớt gánh nặng đó thông qua cơ chế tự động cấu hình.

Ưu điểm nổi bật của Spring Boot là khả năng khởi tạo dự án nhanh, tích hợp sẵn embedded server như Tomcat, quản lý dependency thuận tiện qua Maven hoặc Gradle và dễ dàng phát triển REST API. Trong đề tài này, cả ba service đều được khởi tạo bằng Spring Boot và bổ sung các dependency khác nhau tùy theo chức năng. Inventory Service cần Spring Web, Spring Data JPA, Eureka Client, Actuator, Micrometer, Prometheus và Zipkin. Order Service cần thêm Spring Cloud Circuit Breaker, Spring Kafka. Product Service yêu cầu Spring Web, Spring Data và Lombok.

Một điểm mạnh khác là Spring Boot phù hợp với việc xây dựng service độc lập. Mỗi microservice có thể là một dự án Spring Boot riêng, có file cấu hình riêng, chạy trên cổng riêng và triển khai như một tiến trình riêng biệt. Điều này rất phù hợp với tư duy kiến trúc microservices.

2.3. RESTful API

RESTful API là hình thức giao tiếp phổ biến giữa client và server hoặc giữa các service với nhau trong hệ thống phân tán. REST dựa trên giao thức HTTP và sử dụng các phương thức chuẩn như GET, POST, PUT, DELETE để thể hiện hành động thao tác lên tài nguyên.

Trong hệ thống của đề tài, Product Service cung cấp API POST /api/product để tạo sản phẩm và GET /api/product để lấy danh sách sản phẩm. Inventory Service cung cấp API GET /api/inventory để kiểm tra trạng thái tồn kho theo skuCode. Order Service cung cấp API POST /api/order để đặt hàng. Đây đều là các ví dụ điển hình của REST API, trong đó tài nguyên được biểu diễn qua URL, dữ liệu trao đổi ở dạng JSON và trạng thái xử lý được thể hiện bằng HTTP status code.

RESTful API có nhiều ưu điểm như đơn giản, dễ tích hợp, dễ kiểm thử bằng Postman và phù hợp với giao tiếp giữa các hệ thống không đồng nhất. Tuy nhiên, khi dùng trong môi trường microservices, việc gọi REST API qua mạng cũng làm phát sinh độ trễ và nguy cơ lỗi, vì vậy cần kết hợp với các cơ chế bảo vệ như timeout, retry và circuit breaker.

2.4. Cơ sở dữ liệu trong Microservices

Một nguyên tắc quan trọng khi thiết kế microservices là “database per service”, tức mỗi service có cơ sở dữ liệu riêng. Nguyên tắc này nhằm bảo đảm tính độc lập giữa các service, tránh việc nhiều service cùng phụ thuộc vào một lược đồ dữ liệu chung. Nếu dùng chung một database, dù code đã tách thành nhiều service nhưng bản chất dữ liệu vẫn bị ràng buộc, khiến hệ thống khó bảo trì và khó mở rộng.

Trong đề tài này, Product Service có thể sử dụng MongoDB hoặc một cơ sở dữ liệu phù hợp với Spring Data, trong khi Inventory Service và Order Service sử dụng cơ sở dữ liệu quan hệ thông qua Spring Data JPA. Điều này phản ánh việc lựa chọn công nghệ dựa trên nhu cầu nghiệp vụ cụ thể. Dữ liệu sản phẩm thường linh hoạt về cấu trúc hơn, trong khi dữ liệu kho và đơn hàng cần tính toàn vẹn giao dịch cao hơn.

Việc tách database cũng giúp mỗi service được tối ưu độc lập, dễ backup và dễ scale. Tuy nhiên, nó cũng đặt ra thách thức về đồng bộ dữ liệu giữa các service. Trong đề tài, thay vì chia sẻ trực tiếp dữ liệu, các service giao tiếp với nhau thông qua API hoặc sự kiện Kafka, đây là cách tiếp cận đúng đắn hơn trong microservices.

2.5. Eureka Discovery Server

Trong hệ thống phân tán, các service thường không nên gọi nhau bằng địa chỉ IP hay port cố định, vì khi triển khai thực tế, địa chỉ có thể thay đổi hoặc có nhiều instance của cùng một service. Vì vậy cần một cơ chế service discovery, cho phép các service đăng ký bản thân vào một nơi trung tâm, đồng thời tra cứu địa chỉ của service khác khi cần giao tiếp.

Eureka là một giải pháp service discovery phổ biến trong hệ sinh thái Spring Cloud. Mỗi service khi khởi động sẽ đăng ký với Eureka Server thông qua cấu hình eureka.client.serviceUrl.defaultZone, đồng thời khai báo tên bằng spring.application.name. Theo yêu cầu đề, Inventory Service và Order Service đều phải tích hợp Eureka để có thể được discovery bởi các service khác.

Lợi ích của Eureka là tăng tính linh hoạt trong giao tiếp nội bộ. Order Service có thể gọi Inventory Service theo tên service thay vì địa chỉ cụ thể. Khi Inventory Service có nhiều instance, hệ thống còn có thể kết hợp với cơ chế cân bằng tải để phân phối request hợp lý.

2.6. Resilience4j

Trong microservices, việc một service gọi sang service khác qua mạng là điều rất phổ biến. Tuy nhiên, mạng luôn tiềm ẩn rủi ro như mất kết nối, timeout, service đích bị lỗi hoặc phản hồi chậm. Nếu không có cơ chế bảo vệ, một lỗi nhỏ có thể lan truyền dây chuyền và làm ảnh hưởng toàn hệ thống.

Resilience4j là thư viện hỗ trợ fault tolerance, thường được dùng để bảo vệ các lời gọi giữa service với nhau. Theo yêu cầu của Order Service, Resilience4j được dùng để xử lý lỗi khi gọi Inventory Service, bao gồm các cơ chế circuit breaker, retry và timeout.

Circuit breaker có nhiệm vụ "ngắt mạch" tạm thời nếu số lần lỗi vượt ngưỡng, tránh việc liên tục gửi request đến một service đang hỏng. Retry cho phép thử lại một số lần trong trường hợp lỗi tạm thời. Timeout giới hạn thời gian chờ phản hồi, tránh tình trạng treo lâu gây nghẽn luồng xử lý. Việc áp dụng Resilience4j trong Order Service không chỉ giúp tăng tính ổn định mà còn thể hiện tư duy thiết kế phù hợp với hệ thống phân tán thực tế.

2.7. Kafka

Kafka là một nền tảng message broker hỗ trợ truyền dữ liệu theo kiểu publish-subscribe. Thay vì các service phải gọi trực tiếp nhau trong mọi tình huống, Kafka cho phép một service phát đi sự kiện, còn service khác có thể nhận và xử lý sự kiện đó một cách bất đồng bộ.

Theo yêu cầu bài toán, khi đơn hàng được đặt thành công, Order Service phải gửi thông báo đến Kafka với topic notificationTopic. Điều này có ý nghĩa rất quan trọng về mặt kiến trúc. Nếu sau này hệ thống cần thêm Notification Service, Email Service hay Analytics Service, các service này chỉ cần subscribe vào topic liên quan mà không cần thay đổi logic chính của Order Service.

Giao tiếp qua Kafka giúp giảm ràng buộc giữa các service, tăng khả năng mở rộng và phù hợp với các nghiệp vụ không cần phản hồi ngay lập tức. Trong bài toán hiện tại,

việc phát sinh thông báo sau khi lưu đơn hàng là một ví dụ điển hình của giao tiếp bất đồng bộ.

2.8. Actuator, Prometheus và Zipkin

Giám sát là thành phần không thể thiếu của microservices. Khi hệ thống có nhiều service cùng chạy, nhà phát triển cần biết service nào đang sống, service nào phản hồi chậm, request đang đi qua những đâu và hệ thống đang sử dụng tài nguyên như thế nào.

Spring Boot Actuator cung cấp các endpoint phục vụ theo dõi sức khỏe và số liệu nội bộ của ứng dụng. Trong đề tài, Inventory Service và Order Service cần expose các endpoint như `/actuator/health` và `/actuator/prometheus`. Endpoint health giúp kiểm tra trạng thái sống của service, còn endpoint prometheus giúp Prometheus thu thập metric.

Prometheus là công cụ chuyên dùng để thu thập và lưu trữ metric theo chuỗi thời gian. Khi kết hợp với Grafana, nó có thể tạo ra dashboard trực quan để theo dõi CPU, memory, số lượng request, thời gian phản hồi và nhiều thông số khác.

Zipkin là công cụ hỗ trợ distributed tracing. Trong hệ thống nhiều service, một request có thể đi qua Order Service rồi tới Inventory Service. Zipkin giúp theo dõi toàn bộ đường đi đó, rất hữu ích khi phân tích hiệu năng hoặc tìm lỗi trong luồng xử lý liên service. Đây cũng là lý do Zipkin được yêu cầu tích hợp vào Inventory Service và Order Service.

2.9. Docker và Docker Compose

Docker là nền tảng container hóa ứng dụng, cho phép đóng gói phần mềm cùng với các dependency cần thiết thành một đơn vị chạy độc lập gọi là container. Lợi ích lớn nhất của Docker là bảo đảm ứng dụng chạy nhất quán giữa các môi trường phát triển, kiểm thử và triển khai.

Trong đề tài này, cả ba service đều được yêu cầu tạo Dockerfile để đóng gói ứng dụng. Đồng thời, các service sẽ được chạy cùng với database, Eureka Server, Kafka và các thành phần hỗ trợ khác thông qua Docker Compose.

Docker Compose cho phép định nghĩa toàn bộ hệ thống nhiều container trong một file cấu hình. Nhờ đó, người phát triển có thể khởi động đồng thời Order Service, Product Service, Inventory Service, database, Kafka và Eureka bằng một lệnh duy nhất. Đây là cách làm rất phù hợp trong các dự án microservices ở quy mô nhỏ và trung bình.

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Phân tích yêu cầu bài toán

Đề tài được hình thành từ ba yêu cầu riêng biệt nhưng liên kết chặt chẽ với nhau. Do đó, trước khi tiến hành xây dựng hệ thống, cần phân tích kỹ từng yêu cầu để xác định trách nhiệm của mỗi service và cách phối hợp giữa chúng.

3.1.1. Yêu cầu đối với Product Service

Product Service có nhiệm vụ quản lý thông tin sản phẩm. Theo đề bài, service này cần hỗ trợ ít nhất hai API cơ bản: API tạo sản phẩm mới thông qua phương thức POST /api/product, và API lấy danh sách tất cả sản phẩm thông qua GET /api/product. Dữ liệu của sản phẩm gồm các trường id, name, description, price.

Từ yêu cầu này có thể thấy Product Service đóng vai trò là nguồn dữ liệu chính về sản phẩm trong hệ thống. Mọi thông tin cơ bản như tên sản phẩm, mô tả, giá bán phải được lưu trữ và truy xuất từ đây. Trong thực tế, Product Service có thể phát triển thêm các chức năng khác như cập nhật, xóa, phân loại sản phẩm hay tìm kiếm theo từ khóa, nhưng trong phạm vi đề tài, hai API nêu trên là bắt buộc và đủ để minh họa service quản lý danh mục.

3.1.2. Yêu cầu đối với Inventory Service

Inventory Service chịu trách nhiệm quản lý thông tin kho hàng và trả lời câu hỏi sản phẩm còn hàng hay không. Theo đề bài, Inventory Service phải cung cấp API GET /api/inventory với query parameter skuCode, trong đó có thể truyền vào danh sách các mã sản phẩm cần kiểm tra. Kết quả trả về là danh sách đối tượng chứa skuCode và isInStock. Mô hình dữ liệu của Inventory gồm id, skuCode, quantity.

Ngoài phần chức năng nghiệp vụ, Inventory Service còn có thêm các yêu cầu về hạ tầng như đăng ký với Eureka, tích hợp Actuator, Prometheus, Zipkin và viết unit test, integration test với mức coverage tối thiểu 80%. Như vậy, đây không chỉ là service kiểm tra kho đơn thuần mà còn là một thành phần phải đáp ứng tương đối đầy đủ các tiêu chí kỹ thuật của một microservice hoàn chỉnh.

3.1.3. Yêu cầu đối với Order Service

Order Service là service có mức độ phức tạp cao nhất trong ba service, vì nó không chỉ xử lý dữ liệu nội bộ mà còn tương tác với Inventory Service và Kafka. Theo đề bài, Order Service phải hỗ trợ API POST /api/order, nhận danh sách các mặt hàng cần đặt trong orderLineItemsDtoList. Mỗi phần tử gồm skuCode, price, quantity. Nếu các sản phẩm đều còn hàng, hệ thống phải lưu đơn hàng và trả về kết quả thành công. Nếu có sản phẩm không còn trong kho, hệ thống phải từ chối tạo đơn.

Bên cạnh đó, Order Service còn phải gửi thông báo đến Kafka topic notificationTopic khi đặt hàng thành công, đồng thời tích hợp Resilience4j để xử lý lỗi khi gọi Inventory

Service. Service này cũng phải đăng ký với Eureka và tích hợp Actuator, Prometheus, Zipkin. Tương tự Inventory Service, Order Service được yêu cầu có unit test và integration test, với các kịch bản thành công và thất bại do hết hàng.

3.2. Phân tích mối quan hệ giữa các service

Dựa trên yêu cầu chức năng, có thể thấy Product Service hoạt động tương đối độc lập so với hai service còn lại. Product Service chủ yếu phục vụ việc quản lý dữ liệu sản phẩm và chưa được đề bài yêu cầu giao tiếp trực tiếp với Inventory Service hay Order Service.

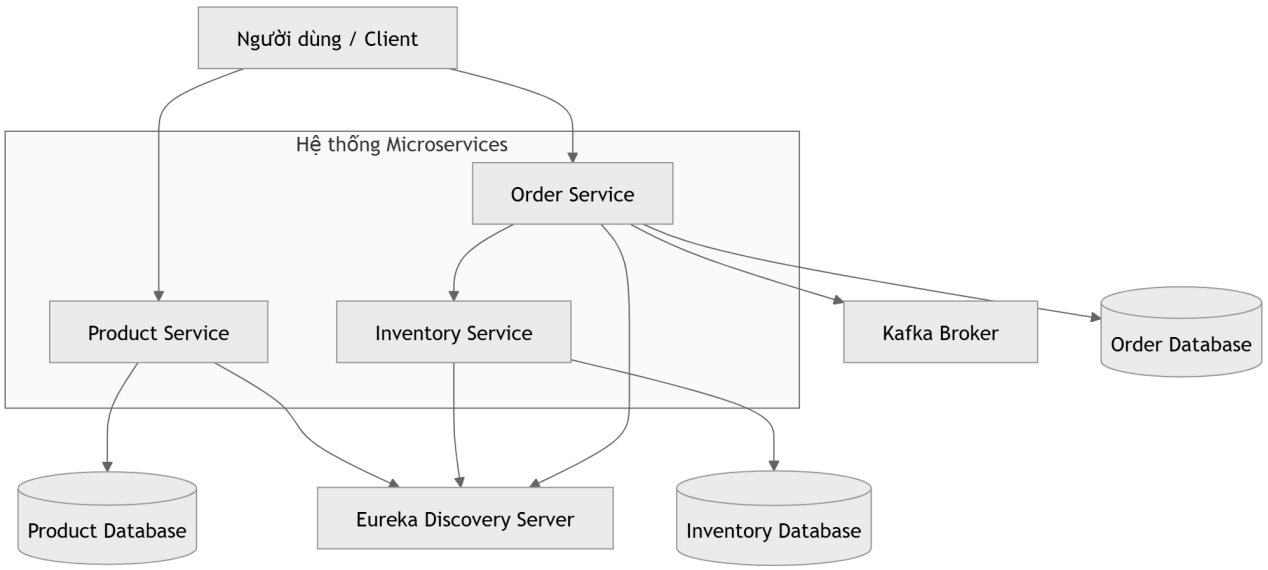
Inventory Service đóng vai trò service hỗ trợ. Bản thân nó không tạo ra đơn hàng, không tạo sản phẩm, nhưng lại rất quan trọng trong quy trình xử lý đặt hàng. Order Service sẽ gọi đến Inventory Service để xác minh rằng các sản phẩm trong đơn đặt hàng có đủ số lượng trong kho hay không. Nói cách khác, Inventory Service là một service nghiệp vụ phụ thuộc của Order Service.

Order Service là trung tâm của quy trình nghiệp vụ đặt hàng. Nó nhận dữ liệu đầu vào từ client, kiểm tra điều kiện với Inventory Service, lưu đơn hàng và phát sự kiện ra Kafka. Như vậy, luồng nghiệp vụ chính của hệ thống được xây dựng xoay quanh Order Service. Từ đó, trong thiết kế tổng thể, cần đặt Order Service ở vị trí phối hợp với các service khác, thay vì hoạt động tách rời.

3.3. Thiết kế kiến trúc tổng thể

Kiến trúc tổng thể của hệ thống được tổ chức theo mô hình microservices. Mỗi service được triển khai như một ứng dụng Spring Boot độc lập. Các service được đăng ký với Eureka Server để hỗ trợ discovery. Product Service, Inventory Service và Order Service đều có kho dữ liệu riêng, giúp bảo đảm tính độc lập. Order Service giao tiếp đồng bộ với Inventory Service qua REST API và giao tiếp bất đồng bộ với Kafka khi phát sinh sự kiện đặt hàng thành công.

Về mặt triển khai, các thành phần đều có thể được container hóa bằng Docker. Khi triển khai đồng thời, Docker Compose sẽ chịu trách nhiệm khởi tạo các container cho các service, database, Kafka và Eureka Server. Kiến trúc này có ưu điểm là rõ ràng, dễ mở rộng, đồng thời bám sát yêu cầu thực tế của các hệ thống thương mại điện tử.



3.4. Phân tích luồng nghiệp vụ

3.4.1. Luồng thêm sản phẩm

Quy trình thêm sản phẩm là quy trình đơn giản nhất trong hệ thống. Client gửi yêu cầu HTTP POST đến endpoint /api/product kèm theo dữ liệu sản phẩm gồm tên, mô tả và giá. Product Service tiếp nhận dữ liệu, ánh xạ sang entity hoặc model, rồi lưu xuống database. Sau đó service trả về mã trạng thái 201 Created để xác nhận rằng tài nguyên mới đã được tạo thành công. Quy trình này phản ánh vai trò quản lý tài nguyên của Product Service.

3.4.2. Luồng lấy danh sách sản phẩm

Khi client muốn xem danh sách sản phẩm, một request GET được gửi đến /api/product. Product Service truy vấn database, lấy toàn bộ danh sách sản phẩm, ánh xạ sang dạng dữ liệu trả về rồi gửi lại cho client. Luồng này không yêu cầu tương tác với các service khác và do đó có độ phức tạp thấp.

3.4.3. Luồng kiểm tra tồn kho

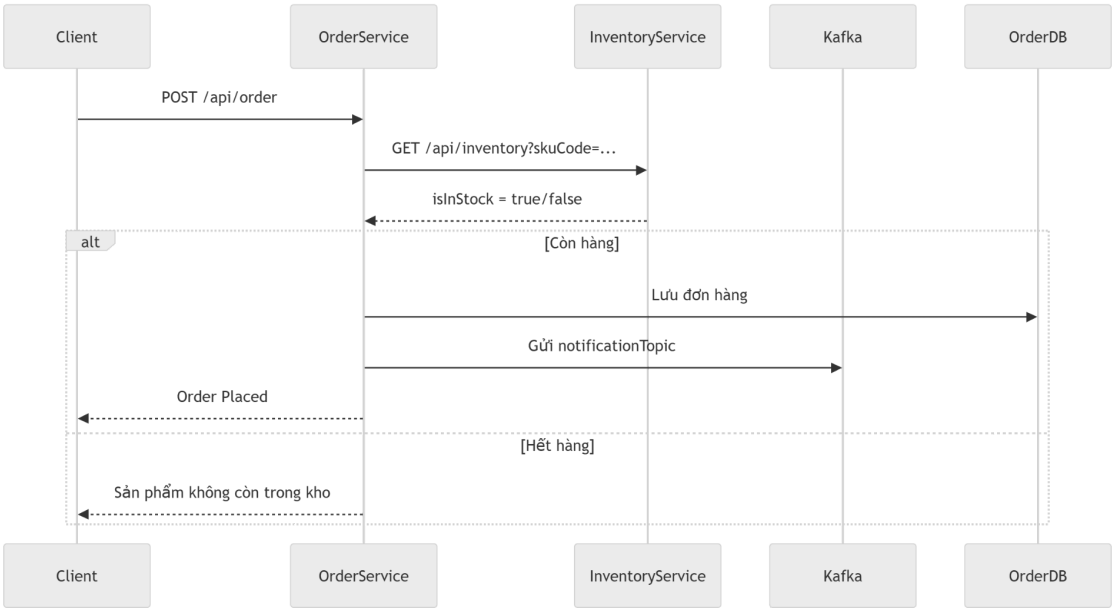
Luồng kiểm tra tồn kho có thể được kích hoạt trực tiếp bởi client hoặc gián tiếp bởi Order Service. Client hoặc service gọi sang Inventory Service bằng API GET /api/inventory và truyền danh sách skuCode. Inventory Service sau đó truy vấn dữ liệu kho, xác định sản phẩm nào có quantity > 0, rồi trả về danh sách kết quả với từng skuCode và trạng thái isInStock. Đây là luồng nghiệp vụ quan trọng vì nó là điều kiện tiền đề cho việc tạo đơn hàng.

3.4.4. Luồng đặt hàng

Đây là luồng nghiệp vụ chính của hệ thống và có mức độ phức tạp cao nhất. Đầu tiên, client gửi request POST đến /api/order cùng danh sách sản phẩm muốn mua. Order Service tiếp nhận dữ liệu, tạo đối tượng đơn hàng và sinh mã đơn hàng. Trước khi lưu,

Order Service phải lấy ra danh sách skuCode từ request và gọi Inventory Service để kiểm tra tồn kho.

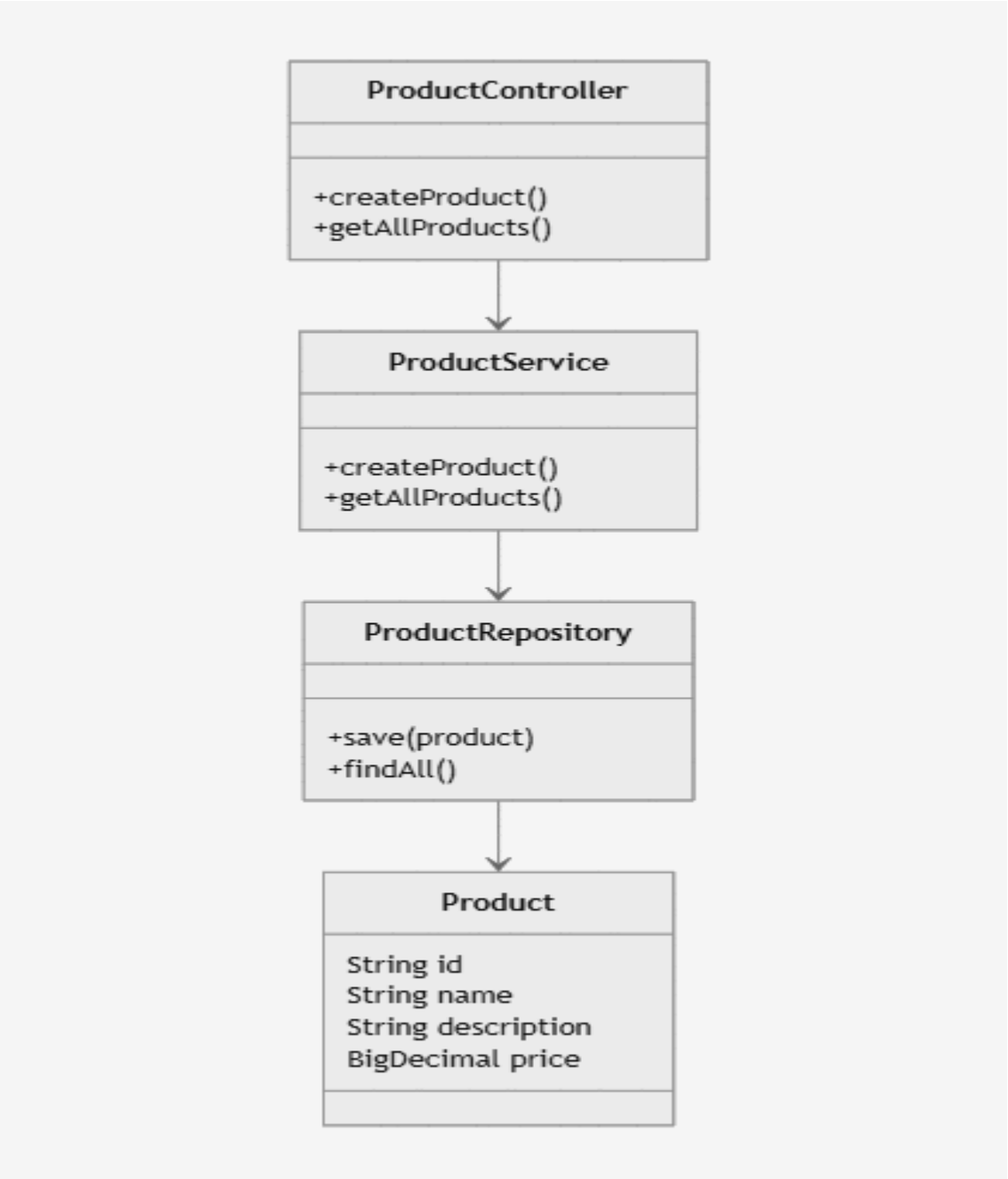
Nếu Inventory Service trả về kết quả cho biết tất cả các sản phẩm đều còn hàng, Order Service mới tiến hành lưu đơn hàng xuống database. Sau khi lưu thành công, service sẽ gửi một sự kiện đến Kafka topic notificationTopic, ví dụ phục vụ thông báo đơn hàng thành công hoặc các xử lý hậu kỳ khác. Cuối cùng, service trả về trạng thái thành công cho client. Nếu Inventory Service cho biết một hay nhiều sản phẩm không còn hàng, Order Service sẽ dừng quy trình và trả về lỗi, không lưu đơn hàng.



3.5. Thiết kế dữ liệu

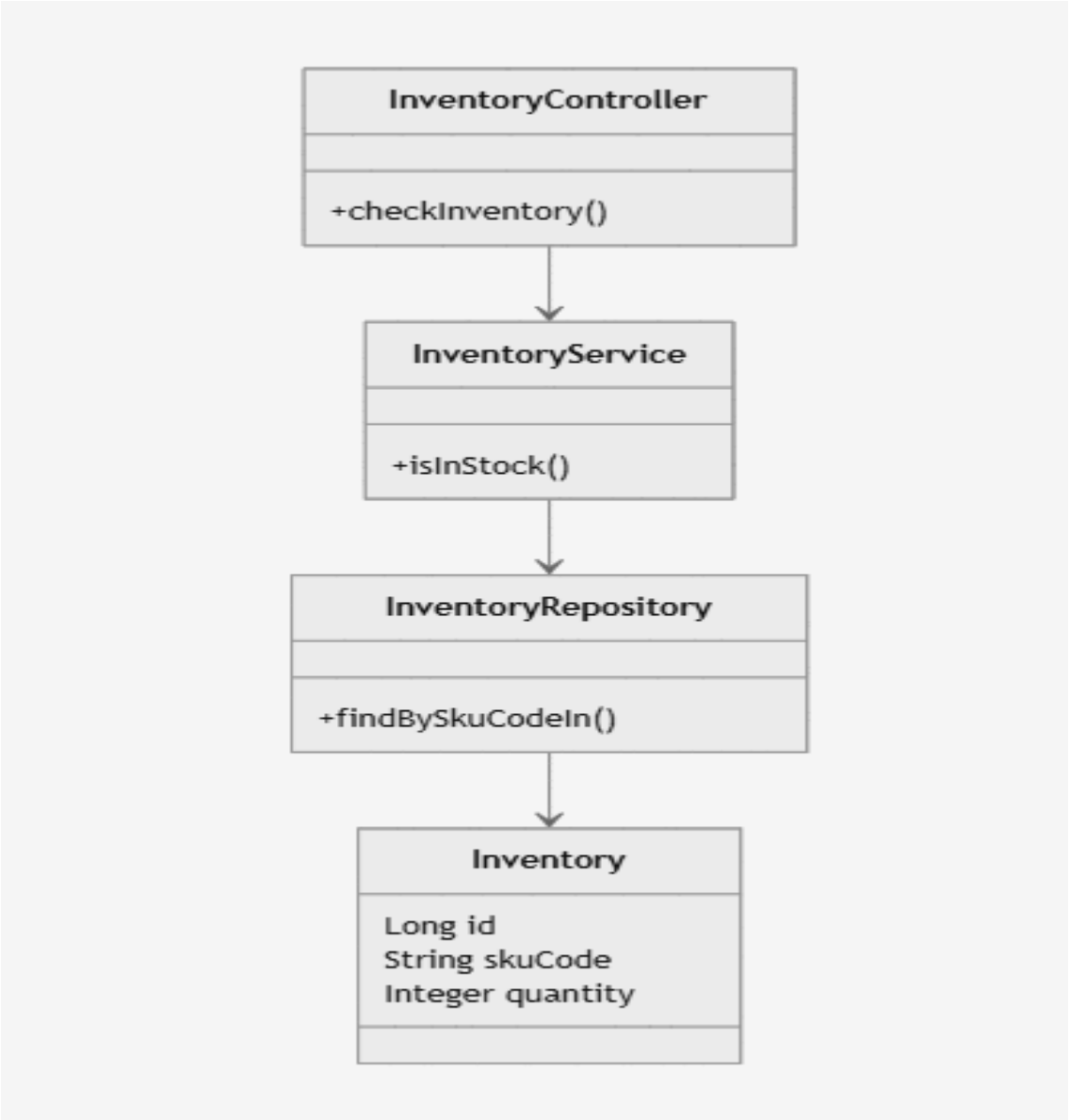
3.5.1. Dữ liệu của Product Service

Dữ liệu sản phẩm được tổ chức quanh bốn trường chính là id, name, description, price. Trong đó, id dùng để định danh duy nhất một sản phẩm; name thể hiện tên sản phẩm; description lưu mô tả; price là giá bán. Mô hình này tuy đơn giản nhưng đủ để biểu diễn một danh mục sản phẩm cơ bản.



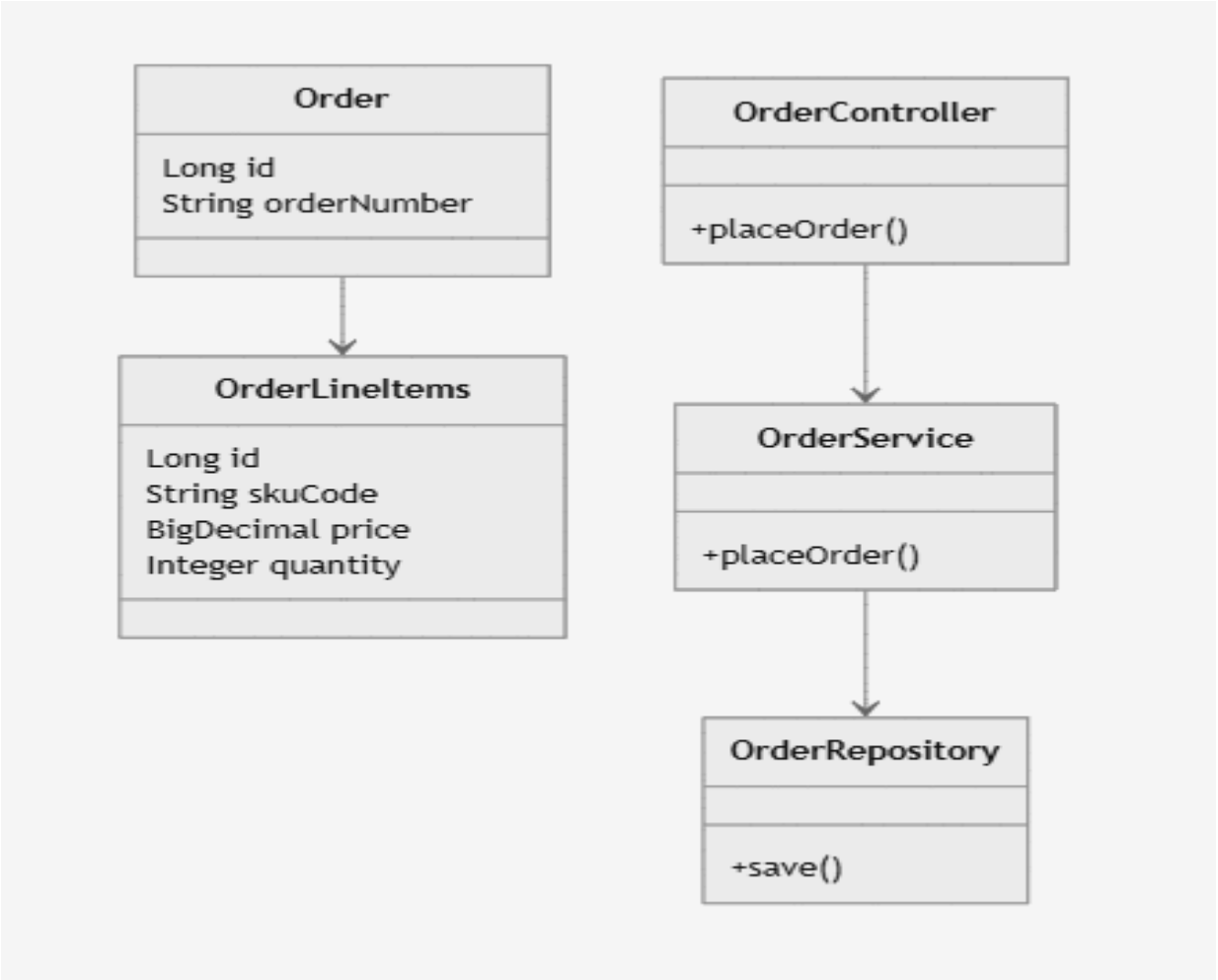
3.5.2. Dữ liệu của Inventory Service

Dữ liệu tồn kho được thiết kế với ba trường: id, skuCode, quantity. Trường skuCode là mã sản phẩm dùng để liên kết với nghiệp vụ đặt hàng, trong khi quantity phản ánh số lượng còn lại trong kho. Thiết kế này cho phép truy vấn nhanh theo mã sản phẩm và xác định chính xác khả năng cung ứng của từng mặt hàng.



3.5.3. Dữ liệu của Order Service

Order Service có hai đối tượng dữ liệu chính là Order và OrderLineItems. Order chứa id và orderNumber, đại diện cho một đơn hàng tổng quát. OrderLineItems chứa id, skuCode, price, quantity, đại diện cho từng dòng sản phẩm trong đơn hàng. Quan hệ giữa hai bảng là một-nhiều, trong đó một đơn hàng có thể chứa nhiều dòng sản phẩm. Thiết kế này rất phù hợp với nghiệp vụ đặt hàng thực tế.



3.6. Thiết kế lớp và cấu trúc dự án

Mỗi service trong hệ thống đều được tổ chức theo mô hình phân lớp phổ biến của Spring Boot, gồm controller, service, repository, model hoặc entity, dto và cấu hình. Điều này giúp tách biệt rõ từng trách nhiệm trong mã nguồn.

Theo cấu trúc tham khảo trong đề bài, Product Service gồm các thư mục như controller, model, repository, service, dto, util, cùng file khởi động ứng dụng và thư mục resources. Inventory Service có cấu trúc tương tự, còn Order Service bổ sung thêm listener và config, phản ánh việc service này cần làm việc với Kafka và các thành phần cấu hình khác.

CHƯƠNG 4. XÂY DỰNG HỆ THỐNG

4.1. Môi trường và công nghệ sử dụng

Để hiện thực hệ thống microservices, đề tài sử dụng Java làm ngôn ngữ lập trình chính do đây là ngôn ngữ được hỗ trợ mạnh trong hệ sinh thái Spring. Framework trung tâm là Spring Boot, kết hợp với các thành phần Spring Data, Spring Cloud và các thư viện hỗ trợ khác.

Công cụ phát triển có thể là IntelliJ IDEA hoặc Visual Studio Code. Việc quản lý phụ thuộc được thực hiện thông qua Maven. Quá trình kiểm thử API sử dụng Postman. Về triển khai, Docker được sử dụng để đóng gói service thành container, còn Docker Compose được dùng để chạy đồng thời nhiều thành phần.

Công nghệ cụ thể theo từng service như sau:

- **Product Service** dùng Spring Web, Spring Data và Lombok.
- **Inventory Service** dùng Spring Web, Spring Data JPA, Eureka Client, Lombok, Spring Boot Actuator, Micrometer Tracing, Micrometer Registry Prometheus.
- **Order Service** dùng Spring Web, Spring Data JPA, Eureka Client, Spring Cloud Circuit Breaker, Spring Kafka, Lombok, Spring Boot Actuator, Micrometer Tracing và Micrometer Registry Prometheus.

4.2. Xây dựng Product Service

4.2.1. Vai trò của Product Service

Product Service đóng vai trò quản lý danh mục sản phẩm của hệ thống. Mặc dù trong đề tài chức năng của service này tương đối cơ bản, nhưng đây là thành phần rất quan trọng vì mọi nghiệp vụ liên quan đến sản phẩm đều phải dựa trên dữ liệu mà service này cung cấp. Nếu Product Service không tồn tại, hệ thống sẽ thiếu nguồn dữ liệu chuẩn về tên, mô tả và giá sản phẩm.

4.2.2. Thiết kế API của Product Service

Product Service cung cấp hai API chính. API thứ nhất là `POST /api/product` dùng để tạo sản phẩm mới. Client gửi request body chứa `name`, `description` và `price`. Service tiếp nhận dữ liệu, tạo đối tượng sản phẩm và lưu xuống cơ sở dữ liệu. Kết quả trả về là mã trạng thái HTTP 201 để xác nhận rằng sản phẩm đã được tạo thành công.

API thứ hai là `GET /api/product` dùng để lấy danh sách toàn bộ sản phẩm. Service sẽ truy vấn dữ liệu từ database và trả về cho client ở dạng JSON. Theo đề bài, đây là hai API cốt lõi mà Product Service phải đáp ứng.

4.2.3. Mô hình dữ liệu của Product Service

Mô hình sản phẩm bao gồm bốn trường: id, name, description, price. Khi hiện thực, id có thể được sinh tự động, name và description là kiểu chuỗi, còn price nên dùng kiểu số thập phân để bảo đảm độ chính xác trong tính toán giá tiền. Thiết kế này đủ để phản ánh dữ liệu cơ bản của sản phẩm.

4.2.4. Tổ chức mã nguồn Product Service

Mã nguồn Product Service được chia thành các lớp chính. Controller nhận request HTTP và trả response. Service chứa logic xử lý nghiệp vụ, ví dụ tạo sản phẩm và lấy danh sách sản phẩm. Repository chịu trách nhiệm giao tiếp với cơ sở dữ liệu. Model hoặc entity biểu diễn cấu trúc dữ liệu sản phẩm. DTO có thể được sử dụng để tách biệt dữ liệu đầu vào, đầu ra với dữ liệu lưu trữ.

Nhờ cách tổ chức này, mã nguồn trở nên rõ ràng, dễ đọc và thuận tiện mở rộng trong tương lai. Nếu sau này cần bổ sung chức năng cập nhật hoặc xóa sản phẩm, nhà phát triển chỉ cần mở rộng thêm các phương thức tương ứng trong từng lớp.

4.3. Xây dựng Inventory Service

4.3.1. Vai trò của Inventory Service

Inventory Service là thành phần quản lý thông tin kho hàng. Trong hệ thống bán hàng, kiểm tra tồn kho là bước không thể thiếu trước khi xác nhận đơn hàng. Nếu không có service này, hệ thống có thể cho phép người dùng đặt những sản phẩm đã hết hàng, dẫn đến sai lệch dữ liệu và trải nghiệm không tốt.

Inventory Service được thiết kế như một service độc lập để đảm nhiệm duy nhất trách nhiệm kiểm tra và quản lý tồn kho. Đây là cách làm đúng với nguyên tắc single responsibility của microservices.

4.3.2. Thiết kế API kiểm tra tồn kho

Theo yêu cầu, Inventory Service cung cấp API GET /api/inventory với query parameter là danh sách skuCode. Khi nhận request, service sẽ truy vấn database để tìm các bản ghi kho tương ứng với các mã sản phẩm này, sau đó xác định sản phẩm nào còn hàng dựa trên quantity > 0, rồi trả về kết quả gồm skuCode và isInStock.

Thiết kế trả về danh sách thay vì một giá trị duy nhất là lựa chọn hợp lý vì một đơn hàng có thể chứa nhiều sản phẩm. Nhờ đó, Order Service có thể gửi danh sách mã sản phẩm cần kiểm tra trong một lần gọi, giúp giảm số lượng request nội bộ và tăng hiệu quả xử lý.

4.3.3. Mô hình dữ liệu Inventory

Mô hình dữ liệu của Inventory Service gồm ba trường là id, skuCode, quantity. Trong triển khai thực tế, skuCode nên được đánh chỉ mục để việc tra cứu theo mã sản phẩm

nhanh hơn. Trường quantity cần bảo đảm không âm, vì số lượng âm không có ý nghĩa về mặt nghiệp vụ.

4.3.4. Sử dụng Spring Data JPA

Inventory Service được yêu cầu dùng Spring Data JPA để thao tác với cơ sở dữ liệu quan hệ. Điều này cho phép ánh xạ entity sang bảng dữ liệu và tự động sinh nhiều phương thức thao tác thông qua repository. Một repository điển hình có thể hỗ trợ tìm danh sách kho theo danh sách skuCode, giúp service triển khai logic kiểm tra tồn kho một cách ngắn gọn và dễ bảo trì.

4.3.5. Tích hợp Eureka trong Inventory Service

Theo đề bài, Inventory Service phải đăng ký với Eureka thông qua eureka.client.serviceUrl.defaultZone và đặt tên service là inventory-service. Nhờ cơ chế này, các service khác, đặc biệt là Order Service, có thể discovery Inventory Service mà không cần biết trước địa chỉ cụ thể. Đây là một bước rất quan trọng trong quá trình biến Inventory Service thành một microservice thực thụ, chứ không chỉ là một ứng dụng REST API thông thường.

4.3.6. Tích hợp Actuator, Prometheus và Zipkin

Inventory Service được yêu cầu expose endpoint /actuator/health và /actuator/prometheus, đồng thời sử dụng Micrometer để tích hợp với Prometheus và Zipkin. Điều này giúp service dễ dàng được giám sát khi triển khai thực tế. Khi có lỗi hoặc hiệu năng giảm, nhà phát triển có thể nhanh chóng xác định trạng thái và nguyên nhân.

4.3.7. Kiểm thử Inventory Service

Inventory Service phải có unit test và integration test, bao gồm ít nhất các trường hợp kiểm tra tồn kho thành công và kiểm tra với sản phẩm không tồn tại. Đề bài cũng yêu cầu test coverage tối thiểu 80%. Điều này cho thấy service không chỉ cần hoạt động đúng mà còn phải được kiểm chứng một cách hệ thống.

4.4. Xây dựng Order Service

4.4.1. Vai trò của Order Service

Order Service là trung tâm xử lý nghiệp vụ đặt hàng của hệ thống. Nếu Product Service quản lý dữ liệu sản phẩm và Inventory Service quản lý số lượng hàng tồn, thì Order Service là nơi kết hợp các thông tin đó để tạo ra một giao dịch hoàn chỉnh. Đây là service có mức độ phức tạp cao nhất vì liên quan đồng thời đến lưu trữ dữ liệu, gọi sang service khác và phát sinh sự kiện.

4.4.2. Thiết kế API đặt hàng

Order Service cung cấp API POST /api/order. Request body chứa danh sách orderLineItemsDtoList, trong đó mỗi phần tử gồm skuCode, price, quantity. Khi nhận request, service cần kiểm tra dữ liệu đầu vào, ánh xạ sang đối tượng nghiệp vụ, sinh mã đơn hàng rồi bắt đầu quy trình xử lý. Nếu thành công, service trả về HTTP 201 cùng thông báo "Order Placed".

4.4.3. Mô hình dữ liệu Order

Mô hình dữ liệu của Order Service gồm hai thực thể. Thực thể thứ nhất là Order, chứa id và orderNumber. Thực thể thứ hai là OrderLineItems, chứa id, skuCode, price, quantity. Order có quan hệ chứa nhiều OrderLineItems, phản ánh đúng thực tế một đơn hàng có thể gồm nhiều sản phẩm khác nhau.

4.4.4. Luồng xử lý đặt hàng chi tiết

Khi nhận request đặt hàng, Order Service sẽ thực hiện các bước sau. Trước hết, dữ liệu từ request được chuyển thành các đối tượng dòng đơn hàng. Sau đó service sinh một mã đơn hàng duy nhất để gắn vào thực thể Order. Tiếp theo, service lấy toàn bộ skuCode từ các dòng đơn hàng và gọi Inventory Service để kiểm tra tình trạng tồn kho.

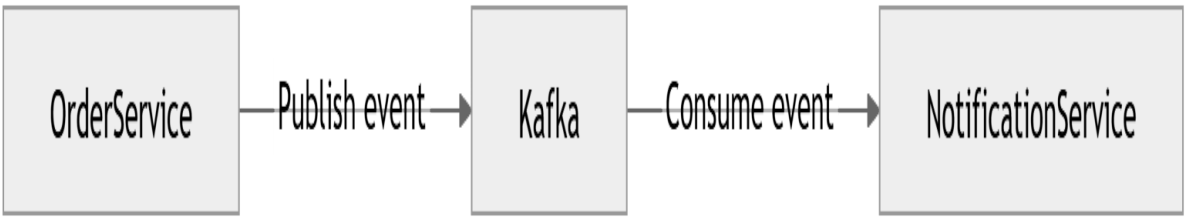
Nếu kết quả từ Inventory Service cho thấy tất cả sản phẩm đều còn hàng, Order Service sẽ lưu đơn hàng vào database. Sau khi lưu thành công, service phát đi một message lên Kafka topic notificationTopic. Cuối cùng, client nhận về thông báo thành công. Ngược lại, nếu một sản phẩm bất kỳ không còn hàng, Order Service sẽ hủy bỏ thao tác lưu và trả về lỗi cho client. Đây chính là điểm thể hiện tính nhất quán nghiệp vụ của hệ thống.

4.4.5. Tích hợp Eureka trong Order Service

Tương tự Inventory Service, Order Service phải đăng ký vào Eureka với tên order-service. Nhờ đó, khi cần gọi Inventory Service, service có thể tra cứu thông qua registry thay vì phụ thuộc vào địa chỉ cứng. Đây là nền tảng để triển khai hệ thống linh hoạt hơn khi scale nhiều instance.

4.4.6. Tích hợp Kafka trong Order Service

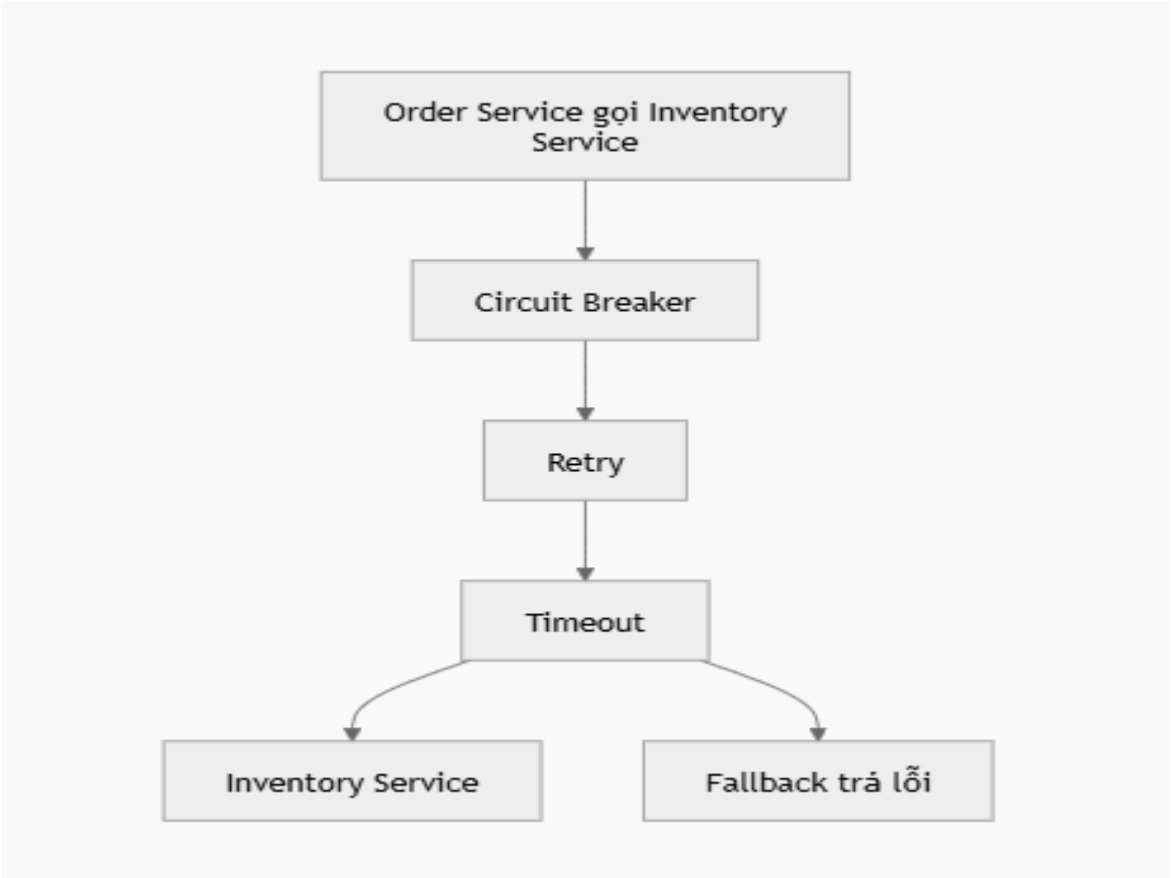
Một trong những yêu cầu đặc trưng của Order Service là gửi thông báo đến Kafka khi đơn hàng được đặt thành công. Theo đề bài, topic được sử dụng là notificationTopic. Trong triển khai, Order Service sẽ đóng vai trò producer. Sau khi hoàn tất giao dịch lưu đơn hàng, service gửi message chứa thông tin đơn hàng hoặc thông báo liên quan. Việc phát sự kiện sau khi đặt hàng thành công mở ra khả năng mở rộng hệ thống theo hướng event-driven.



4.4.7. Tích hợp Resilience4j trong Order Service

Vì Order Service phải gọi đến Inventory Service qua mạng, nên nó phải đối mặt với nguy cơ lỗi kết nối, timeout hoặc service đích ngừng hoạt động. Đề bài yêu cầu dùng Resilience4j để xử lý các tình huống này, bao gồm circuit breaker, retry và timeout.

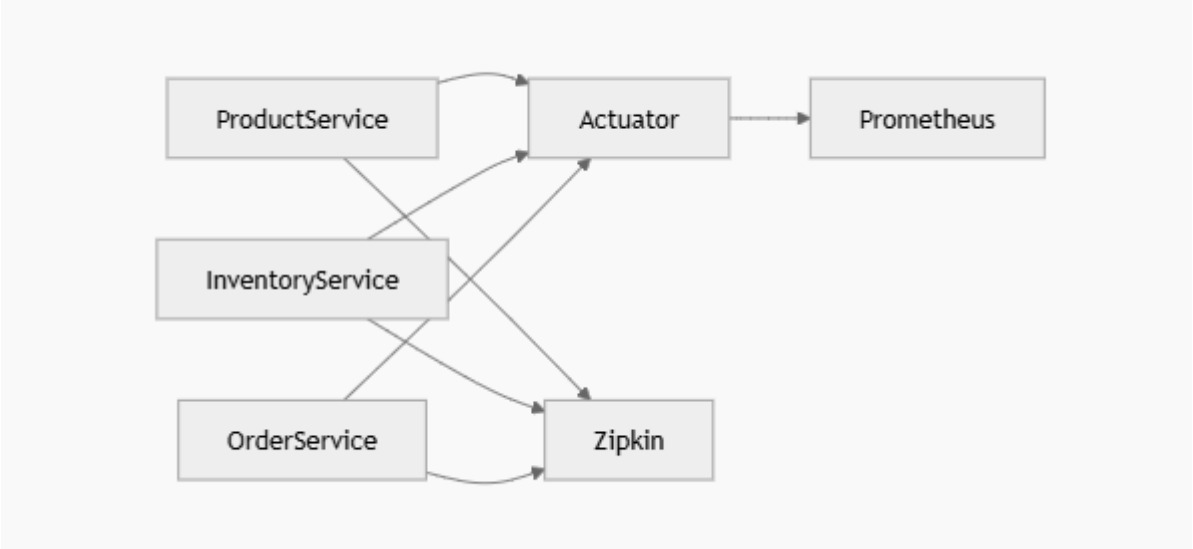
Khi Inventory Service lỗi liên tục, circuit breaker sẽ mở mạch và ngăn Order Service tiếp tục gửi request sang, từ đó tránh làm hệ thống tắc nghẽn. Retry được sử dụng khi lỗi chỉ là tạm thời, ví dụ mất kết nối ngắn hạn. Timeout giúp Order Service không chờ quá lâu, bảo đảm phản hồi cho người dùng trong thời gian hợp lý. Việc tích hợp ba cơ chế này thể hiện cách tiếp cận trưởng thành trong thiết kế microservices.



4.4.8. Tích hợp Actuator, Prometheus, Zipkin

Tương tự Inventory Service, Order Service được yêu cầu expose endpoint health và prometheus, đồng thời sử dụng tracing để theo dõi luồng xử lý. Việc này rất cần thiết

vì Order Service là nơi kết hợp nhiều thao tác nhất, nếu không có monitoring và tracing thì rất khó phát hiện nút thắt hiệu năng hay lỗi trong giao tiếp với các thành phần khác.

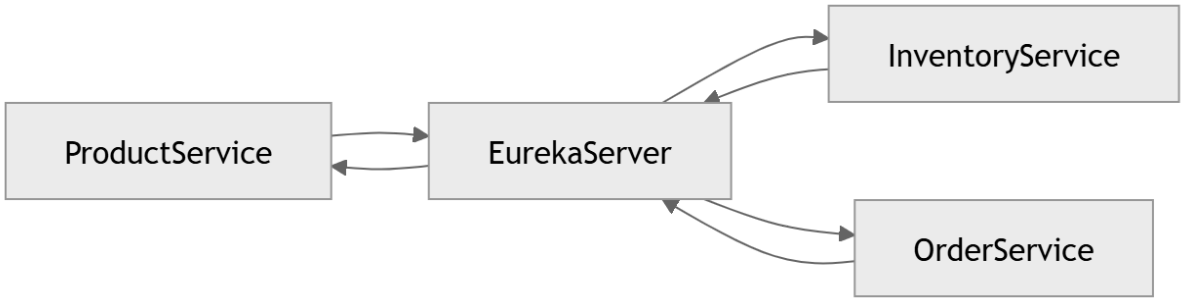


4.4.9. Kiểm thử Order Service

Đề bài yêu cầu viết test cho ít nhất hai trường hợp: đặt hàng thành công và đặt hàng thất bại do sản phẩm không có trong kho, đồng thời bảo đảm coverage tối thiểu 80%. Khi xây dựng test, cần mô phỏng kết quả từ Inventory Service để kiểm tra đúng logic của Order Service trong từng nhánh xử lý. Ngoài ra, có thể mở rộng thêm test cho trường hợp Kafka producer được gọi sau khi lưu đơn thành công.

4.5. Tích hợp Eureka Discovery Server

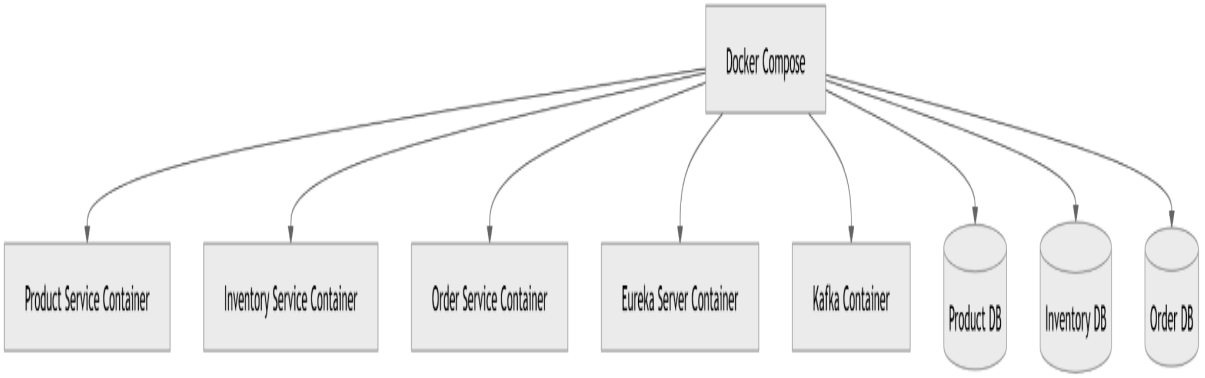
Việc sử dụng Eureka cho phép các service tự động đăng ký và tìm thấy nhau trong hệ thống. Trong triển khai, mỗi service cần khai báo tên riêng và địa chỉ Eureka Server. Khi khởi động, Eureka sẽ lưu thông tin các instance đang hoạt động. Nhờ đó, Order Service có thể tìm Inventory Service theo tên inventory-service, còn các công cụ quản trị có thể quan sát toàn bộ service từ một dashboard chung.



4.6. Docker hóa hệ thống

Việc Docker hóa từng service mang lại nhiều lợi ích. Trước hết, mỗi service có thể được đóng gói cùng toàn bộ dependency vào một image độc lập. Điều này giúp ứng dụng chạy nhất quán ở mọi môi trường. Thứ hai, khi kết hợp với Docker Compose, toàn bộ hệ thống có thể được khởi động chỉ bằng một lệnh duy nhất.

Trong đề tài, cả Product Service, Inventory Service và Order Service đều được yêu cầu tạo Dockerfile. Inventory Service cần chạy cùng database và Eureka Server; Order Service cần chạy cùng database, Eureka Server và Kafka; Product Service cần chạy cùng database tương ứng.



CHƯƠNG 5. KIỂM THỬ VÀ ĐÁNH GIÁ

5.1. Mục tiêu kiểm thử

Mục tiêu của kiểm thử là xác minh rằng từng service hoạt động đúng theo yêu cầu và toàn bộ hệ thống phối hợp chính xác theo luồng nghiệp vụ đã thiết kế. Trong microservices, kiểm thử không chỉ dừng ở việc một API trả về đúng dữ liệu, mà còn cần xét đến khả năng giao tiếp giữa các service, xử lý tình huống lỗi và tính ổn định của toàn hệ thống.

Đối với đề tài này, kiểm thử cần làm rõ ba vấn đề. Thứ nhất, Product Service có thực sự tạo và truy xuất được sản phẩm hay không. Thứ hai, Inventory Service có trả về chính xác trạng thái tồn kho cho từng skuCode hay không. Thứ ba, Order Service có đặt hàng đúng quy trình, chỉ lưu đơn khi còn hàng và phát sinh sự kiện Kafka sau khi thành công hay không.

5.2. Kiểm thử Product Service

Ở Product Service, các trường hợp kiểm thử cơ bản gồm tạo sản phẩm mới và lấy danh sách sản phẩm. Với trường hợp tạo mới, người kiểm thử gửi request POST chứa dữ liệu sản phẩm hợp lệ. Kết quả mong đợi là hệ thống trả về HTTP 201 và dữ liệu được lưu trong database. Với trường hợp lấy danh sách sản phẩm, người kiểm thử gửi GET request tới `/api/product` và mong đợi hệ thống trả về danh sách sản phẩm đúng với dữ liệu đã lưu.

Việc kiểm thử Product Service tuy đơn giản nhưng rất cần thiết, bởi nếu dữ liệu sản phẩm không được quản lý chính xác thì các nghiệp vụ khác trong hệ thống sẽ mất đi nền tảng dữ liệu đầu vào.

5.3. Kiểm thử Inventory Service

Theo yêu cầu đề bài, Inventory Service cần có ít nhất các test cho trường hợp kiểm tra tồn kho thành công và kiểm tra với sản phẩm không tồn tại. Với trường hợp thành công, service phải trả về `isInStock = true` cho sản phẩm có `quantity > 0`. Với trường hợp sản phẩm không tồn tại hoặc không có trong kho, service phải trả về trạng thái phù hợp, thường là `false`.

Bên cạnh đó, cần kiểm thử thêm trường hợp `quantity = 0`, vì đây là một kịch bản nghiệp vụ rất quan trọng. Nếu hệ thống xử lý sai trường hợp này, người dùng có thể đặt hàng cho những sản phẩm đã hết hàng. Ngoài unit test, integration test cũng cần được thực hiện để kiểm tra tương tác thực sự với database.

5.4. Kiểm thử Order Service

Order Service cần được kiểm thử kỹ nhất do có mức độ phức tạp cao. Trường hợp đầu tiên là đặt hàng thành công. Trong trường hợp này, dữ liệu request hợp lệ và Inventory

Service trả về rằng tất cả sản phẩm đều còn hàng. Kết quả mong đợi là Order Service lưu đơn hàng, trả về thông báo thành công và gọi producer để gửi message Kafka.

Trường hợp thứ hai là đặt hàng thất bại do sản phẩm không còn trong kho. Khi đó, Inventory Service trả về ít nhất một skuCode với isInStock = false. Kết quả mong đợi là Order Service không lưu đơn hàng và trả lỗi cho client. Đây là trường hợp bắt buộc phải có theo đề bài.

Ngoài ra, cần xem xét kiểm thử khi Inventory Service phản hồi chậm hoặc lỗi. Trường hợp này giúp xác minh cấu hình Resilience4j đã hoạt động đúng hay chưa. Nếu timeout, retry hay circuit breaker được cấu hình đúng, Order Service sẽ không bị treo vô thời hạn và có thể phản hồi lỗi theo cách kiểm soát được.

5.5. Unit test và Integration test

Unit test có nhiệm vụ kiểm tra từng đơn vị logic nhỏ trong service, ví dụ một phương thức xử lý dữ liệu đầu vào hoặc một nhánh điều kiện trong service layer. Điểm mạnh của unit test là chạy nhanh, cô lập và dễ xác định lỗi. Tuy nhiên, unit test không phản ánh đầy đủ hoạt động thực tế của hệ thống.

Integration test kiểm tra sự phối hợp giữa nhiều thành phần, ví dụ controller, service, repository và database. Trong microservices, integration test đặc biệt quan trọng vì nhiều lỗi chỉ xuất hiện khi các lớp làm việc cùng nhau. Inventory Service và Order Service đều được yêu cầu đạt coverage tối thiểu 80%, cho thấy việc kiểm thử là một phần không thể thiếu trong đánh giá chất lượng hệ thống.

5.6. Đánh giá kết quả đạt được

Sau khi xây dựng và kiểm thử, hệ thống đạt được những kết quả quan trọng sau. Product Service thực hiện đúng việc tạo mới và truy vấn danh sách sản phẩm. Inventory Service kiểm tra được tình trạng tồn kho và trả về dữ liệu theo đúng định dạng yêu cầu. Order Service tiếp nhận yêu cầu đặt hàng, phối hợp với Inventory Service để kiểm tra điều kiện và chỉ lưu đơn khi điều kiện kho được bảo đảm. Ngoài ra, Order Service còn có khả năng phát sinh sự kiện Kafka sau khi đặt hàng thành công.

Về mặt kiến trúc, các service được tách biệt rõ ràng, đúng định hướng microservices. Các thành phần hỗ trợ như Eureka, monitoring và Docker giúp hệ thống tiến gần hơn tới một kiến trúc có thể triển khai thực tế, chứ không dừng lại ở mức mô phỏng bài tập API đơn giản.

5.7. Ưu điểm và hạn chế của hệ thống

- **Ưu điểm đầu tiên** của hệ thống là phân tách rõ ràng các miền nghiệp vụ. Product, Inventory và Order không bị trộn lẫn logic, giúp mã nguồn mạch lạc và dễ phát triển.

- **Ưu điểm thứ hai** là khả năng mở rộng linh hoạt. Nếu trong tương lai số lượng đơn hàng tăng mạnh, chỉ cần scale Order Service mà không cần scale toàn bộ hệ thống.
- **Ưu điểm thứ ba** là hệ thống đã tích hợp các cơ chế quan trọng như service discovery, fault tolerance, messaging và monitoring, đây là những yếu tố rất sát với thực tiễn.

Tuy nhiên, hệ thống vẫn còn những hạn chế nhất định:

- **Trước hết,** số lượng thành phần hạ tầng nhiều hơn so với kiến trúc monolithic, khiến cấu hình và triển khai phức tạp hơn.
- **Thứ hai,** việc gỡ lỗi trong môi trường phân tán luôn khó hơn do request đi qua nhiều service.
- **Thứ ba,** hệ thống hiện mới dừng ở mức backend và chưa có lớp giao diện người dùng hay các nghiệp vụ nâng cao như thanh toán, xác thực người dùng và quản lý đơn hàng hậu kỳ.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết luận

Đề tài "Xây dựng hệ thống Microservices với Product Service, Inventory Service và Order Service" đã đạt được mục tiêu đặt ra là xây dựng một hệ thống backend phân tán gồm ba dịch vụ cốt lõi của bài toán bán hàng. Thông qua việc xây dựng Product Service, đề tài đã hiện thực được nghiệp vụ quản lý thông tin sản phẩm. Thông qua Inventory Service, đề tài đã hiện thực được nghiệp vụ kiểm tra tồn kho trên cơ sở dữ liệu quan hệ, đồng thời tích hợp các công cụ giám sát cần thiết. Thông qua Order Service, đề tài đã hiện thực được luồng đặt hàng hoàn chỉnh với khả năng kiểm tra kho, lưu đơn hàng, xử lý lỗi và phát sinh thông điệp qua Kafka.

Về mặt học thuật, đề tài giúp người thực hiện hiểu sâu hơn về sự khác biệt giữa kiến trúc monolithic và microservices, nắm được cách phân chia dịch vụ theo nghiệp vụ, cách tổ chức dữ liệu độc lập, cách thiết kế API giữa các thành phần và cách tích hợp các công cụ hỗ trợ trong hệ sinh thái Spring. Về mặt thực hành, đề tài mang lại kinh nghiệm quan trọng trong việc xây dựng service bằng Spring Boot, sử dụng JPA, tích hợp Eureka, Kafka, Resilience4j, Actuator, Prometheus, Zipkin và Docker.

Có thể nói, kết quả của đề tài không chỉ dừng ở mức hoàn thành yêu cầu môn học, mà còn tạo ra một nền thực tế kỹ thuật tốt để tiếp tục phát triển thành một hệ thống bán hàng hoàn chỉnh trong tương lai.

6.2. Hướng phát triển

Mặc dù hệ thống hiện tại đã đáp ứng được các yêu cầu cốt lõi, vẫn còn nhiều hướng phát triển có thể mở rộng thêm để tăng tính hoàn chỉnh và giá trị ứng dụng thực tế:

- **Hướng phát triển đầu tiên** là bổ sung API Gateway. Hiện tại client có thể gọi trực tiếp đến từng service, nhưng trong hệ thống thực tế, nên có một cổng vào thống nhất để thực hiện routing, bảo mật, giới hạn tốc độ và quản lý request tập trung.
- **Hướng phát triển thứ hai** là tích hợp xác thực và phân quyền. Hệ thống hiện chưa có cơ chế đăng nhập, chưa phân biệt vai trò người dùng. Trong tương lai, có thể bổ sung Spring Security kết hợp JWT để bảo vệ API và kiểm soát quyền truy cập.
- **Hướng phát triển thứ ba** là mở rộng hệ sinh thái service. Ví dụ, có thể xây dựng thêm Notification Service để nhận sự kiện từ Kafka và gửi email hoặc tin nhắn cho người dùng sau khi đặt hàng. Cũng có thể xây dựng Payment Service để xử lý thanh toán, Shipment Service để quản lý giao hàng, hoặc Cart Service để quản lý giỏ hàng.
- **Hướng phát triển thứ tư** là triển khai hệ thống trên Kubernetes thay vì Docker Compose, từ đó hỗ trợ tốt hơn cho việc scale, rolling update và quản lý

tài nguyên trong môi trường production. Đây cũng là hướng mở rộng đã được gợi ý trong các đề bài.

- **Hướng phát triển thứ năm** là hoàn thiện quy trình DevOps với CI/CD, tự động kiểm thử và tự động triển khai. Khi kết hợp với monitoring dashboard và log tập trung, hệ thống sẽ trở nên gần hơn với chuẩn triển khai doanh nghiệp.

Cuối cùng, nếu bổ sung thêm giao diện người dùng và các nghiệp vụ liên quan, hệ thống này có thể phát triển thành một ứng dụng bán hàng trực tuyến hoàn chỉnh, trong đó Product Service, Inventory Service và Order Service tiếp tục đóng vai trò là lõi nghiệp vụ quan trọng.